

Programming Fundamentals

Assignment 2

Instructor: UMAR KHALIL

Due Date: June 2nd, 2026

C++ Source Code — All Programs

Program 1: LESCO Electricity Bill Management System

```
#include <iostream>
#include <string>
#include <iomanip>
using namespace std;
string customerName, customerId, customerType;
int metersInstalled;
double slabRates[] = {12.21, 14.53, 31.51, 38.41, 41.62, 43.04, 44.18, 49.10};
void registerCustomer() {
    cout << "\n==== CUSTOMER REGISTRATION =====\n";
    cout << "Enter Customer Name: ";
    cin.ignore();
    getline(cin, customerName);
    cout << "Enter Customer ID: ";
    cin >> customerId;
    cout << "Customer Type (1 = Household, 2 = Commercial): ";
```

```

int typeChoice;
cin >> typeChoice;
customerType = (typeChoice == 1) ? "Household" : "Commercial";
cout << "Number of Meters Installed (1 = First Meter, 2 = Second Meter): ";
cin >> metersInstalled;
cout << "\nRegistration Successful!\n";
}
double calculateConsumptionCharges(int units) {
    double charges = 0;
    if (units <= 100)
        charges = units * slabRates[0];
    else if (units <= 200)
        charges = 100 * slabRates[0] + (units - 100) * slabRates[1];
    else if (units <= 300)
        charges = 100 * slabRates[0] + 100 * slabRates[1] + (units - 200) * slabRates[2];
    else if (units <= 400)
        charges = 100 * slabRates[0] + 100 * slabRates[1] + 100 * slabRates[2] + (units -
300) * slabRates[3];
    else if (units <= 500)
        charges = 100 * slabRates[0] + 100 * slabRates[1] + 100 * slabRates[2] + 100 *
slabRates[3] + (units - 400) * slabRates[4];
    else if (units <= 600)
        charges = 100 * slabRates[0] + 100 * slabRates[1] + 100 * slabRates[2] + 100 *
slabRates[3] + 100 * slabRates[4] + (units - 500) * slabRates[5];
    else if (units <= 700)
        charges = 100 * slabRates[0] + 100 * slabRates[1] + 100 * slabRates[2] + 100 *
slabRates[3] + 100 * slabRates[4] + 100 * slabRates[5] + (units - 600) * slabRates[6];
    else
        charges = 100 * slabRates[0] + 100 * slabRates[1] + 100 * slabRates[2] + 100 *
slabRates[3] + 100 * slabRates[4] + 100 * slabRates[5] + 100 * slabRates[6] + (units -
700) * slabRates[7];
    return charges;
}
double calculateFixedCharges(int units) {
    if (units <= 300) return 0;
    else if (units <= 400) return 200;
    else if (units <= 500) return 400;
    else if (units <= 600) return 600;
    else if (units <= 700) return 800;
    else return 1000;
}
double calculateGST(double totalBill) {
    return totalBill * 0.18;
}
double calculateIncomeTax(double totalBill) {
    if (customerType == "Household")
        return totalBill * 0.10;
    else
        return totalBill * 0.15;
}
double calculateElectricityDuty(double consumptionCharges) {
    return consumptionCharges * 0.015;
}
double calculateNewConnectionCharges() {
    double meterCost = 5000;
    double meterCableCost = 3000;
    double securityCost = 2000;
    double additionalCharges = 0;
    double newConnectionCost = 0;
    if (customerType == "Household") {
        additionalCharges = (metersInstalled == 1) ? 2500 : 5000;
    } else {
        additionalCharges = (metersInstalled == 1) ? 35000 : 70000;
    }
    cout << "\nDo you want a new proper connection? (1 = Yes, 2 = No): ";
    int choice;
    cin >> choice;
}

```

```

        if (choice == 1)
            newConnectionCost = 250000;
        double total = meterCost + meterCableCost + securityCost + additionalCharges +
newConnectionCost;
        cout << fixed << setprecision(2);
        cout << "\n===== NEW CONNECTION CHARGES =====\n";
        cout << "Meter Cost:                Rs. " << meterCost << "\n";
        cout << "Meter Cable Cost:            Rs. " << meterCableCost << "\n";
        cout << "Security Cost:                Rs. " << securityCost << "\n";
        cout << "Additional Charges:          Rs. " << additionalCharges << "\n";
        if (newConnectionCost > 0)
            cout << "New Connection Cost:      Rs. " << newConnectionCost << "\n";
        cout << "-----\n";
        cout << "Total Connection Cost:  Rs. " << total << "\n";
        cout << "===== \n";
        return total;
    }

void displayCustomerDetails() {
    cout << "\n===== CUSTOMER DETAILS =====\n";
    cout << "Name:                " << customerName << "\n";
    cout << "ID:                  " << customerId << "\n";
    cout << "Type:                " << customerType << "\n";
    cout << "Meters:              " << metersInstalled << "\n";
    cout << "===== \n";
}

void displayFinalBill() {
    int units;
    cout << "\nEnter Units Consumed: ";
    cin >> units;
    double consumptionCharges = calculateConsumptionCharges(units);
    double electricityDuty    = calculateElectricityDuty(consumptionCharges);
    double fixedCharges       = calculateFixedCharges(units);
    double meterRent          = 250;
    double tvFee              = 35;
    double subtotal = consumptionCharges + electricityDuty + fixedCharges + meterRent +
tvFee;
    double gst              = calculateGST(subtotal);
    double incomeTax        = calculateIncomeTax(subtotal);
    double totalBill       = subtotal + gst + incomeTax;
    cout << fixed << setprecision(2);
    cout << "\n===== LESCO ELECTRICITY BILL =====\n";
    cout << "Customer Name:  " << customerName << "\n";
    cout << "Customer ID:    " << customerId << "\n";
    cout << "Customer Type:  " << customerType << "\n";
    cout << "Units Consumed: " << units << "\n";
    cout << "Electricity Consumption Charges: Rs. " << consumptionCharges << "\n";
    cout << "Electricity Duty (1.5%):        Rs. " << electricityDuty << "\n";
    cout << "Fixed Charges:                  Rs. " << fixedCharges << "\n";
    cout << "Meter Rent:                     Rs. " << meterRent << "\n";
    cout << "TV Fee:                         Rs. " << tvFee << "\n";
    cout << "GST (18%):                      Rs. " << gst << "\n";
    cout << "Income Tax:                     Rs. " << incomeTax << "\n";
    cout << "-----\n";
    cout << "Total Payable Bill:              Rs. " << totalBill << "\n";
    cout << "===== \n";
}

void displayMenu() {
    int choice;
    do {
        cout << "\n===== LESCO MAIN MENU =====\n";
        cout << "1. Calculate Monthly Electricity Bill\n";
        cout << "2. Apply for New Electricity Connection\n";
        cout << "3. View Customer Details\n";
        cout << "4. Exit Program\n";
        cout << "Enter your choice: ";
        cin >> choice;
        switch (choice) {

```

```

        case 1: displayFinalBill(); break;
        case 2: calculateNewConnectionCharges(); break;
        case 3: displayCustomerDetails(); break;
        case 4: cout << "Thank you for using LESCO system. Goodbye!\n"; break;
        default: cout << "Invalid choice! Please try again.\n";
    }
} while (choice != 4);
}
int main() {
    cout << "=====\n";
    cout << "  LESCO ELECTRICITY MANAGEMENT SYSTEM  \n";
    cout << "=====\n";
    registerCustomer();
    displayMenu();
    return 0;
}

```

Program 2: Restaurant Billing Management System

```

#include <iostream>
#include <string>
#include <iomanip>
using namespace std;
string customerName, contactNumber, orderType;
int numPersons;
string foodItems[] = {"Chicken Burger", "Zinger Burger", "Pizza Small", "Pizza Large",
                     "Chicken Biryani", "BBQ Platter", "Fries", "Cold Drink"};
double foodPrices[] = {450, 550, 900, 1800, 350, 1200, 250, 120};
int totalItems = 8;
int cartQty[8] = {0};
void registerCustomer() {
    cout << "\n==== CUSTOMER REGISTRATION =====\n";
    cout << "Enter Customer Name: ";
    cin.ignore();
    getline(cin, customerName);
    cout << "Enter Contact Number: ";
    cin >> contactNumber;
    cout << "Order Type (1 = Dine-in, 2 = Takeaway): ";
    int typeChoice;
    cin >> typeChoice;
    orderType = (typeChoice == 1) ? "Dine-in" : "Takeaway";
    cout << "Number of Persons: ";
    cin >> numPersons;
    cout << "\nRegistration Successful!\n";
}
void displayFoodMenu() {
    cout << "\n==== RESTAURANT MENU =====\n";
    cout << left << setw(5) << "No." << setw(20) << "Food Item" << "Price\n";
    cout << "-----\n";
    for (int i = 0; i < totalItems; i++) {
        cout << left << setw(5) << (i + 1) << setw(20) << foodItems[i] << "Rs. " <<
        foodPrices[i] << "\n";
    }
    cout << "=====\n";
}
void placeOrder() {
    displayFoodMenu();
    int itemNo, qty;
    char more;
    do {
        cout << "Enter Item Number to Order: ";

```

```

        cin >> itemNo;
        if (itemNo < 1 || itemNo > totalItems) {
            cout << "Invalid item number!\n";
        } else {
            cout << "Enter Quantity: ";
            cin >> qty;
            cartQty[itemNo - 1] += qty;
            cout << qty << "x " << foodItems[itemNo - 1] << " added to cart.\n";
        }
        cout << "Add more items? (y/n): ";
        cin >> more;
    } while (more == 'y' || more == 'Y');
}

double calculateFoodBill() {
    double total = 0;
    for (int i = 0; i < totalItems; i++) {
        total += cartQty[i] * foodPrices[i];
    }
    return total;
}

double calculateServiceCharges(double foodBill) {
    if (orderType == "Dine-in")
        return foodBill * 0.10;
    else
        return foodBill * 0.05;
}

double calculateGST(double amount) {
    return amount * 0.16;
}

double calculateDiscount(double amount) {
    if (amount >= 3000 && amount <= 5000)
        return amount * 0.05;
    else if (amount > 5000 && amount <= 10000)
        return amount * 0.10;
    else if (amount > 10000)
        return amount * 0.15;
    return 0;
}

void displayCustomerDetails() {
    cout << "\n===== CUSTOMER DETAILS =====\n";
    cout << "Name:           " << customerName << "\n";
    cout << "Contact:          " << contactNumber << "\n";
    cout << "Order Type:       " << orderType << "\n";
    cout << "Persons:          " << numPersons << "\n";
    cout << "=====\n";
}

void displayFinalBill() {
    double foodBill = calculateFoodBill();
    double serviceCharges = calculateServiceCharges(foodBill);
    double subtotal = foodBill + serviceCharges;
    double gst = calculateGST(subtotal);
    double discount = calculateDiscount(subtotal + gst);
    bool freeDelivery = (subtotal + gst) > 5000;
    double totalPayable = subtotal + gst - discount;
    cout << fixed << setprecision(2);
    cout << "\n===== RESTAURANT BILL =====\n";
    cout << "Customer Name:    " << customerName << "\n";
    cout << "Order Type:       " << orderType << "\n";
    cout << "Number of Persons: " << numPersons << "\n\n";
    cout << "--- Items Ordered ---\n";
    for (int i = 0; i < totalItems; i++) {
        if (cartQty[i] > 0) {
            cout << foodItems[i] << " x" << cartQty[i] << " = Rs. " << cartQty[i] *
foodPrices[i] << "\n";
        }
    }
    cout << "\n";
}

```

```

        cout << "Food Bill:           Rs. " << foodBill << "\n";
        cout << "Service Charges:   Rs. " << serviceCharges << "\n";
        cout << "GST (16%):           Rs. " << gst << "\n";
        cout << "Discount:           Rs. " << discount << "\n";
        if (freeDelivery)
            cout << "Delivery:           FREE (Bill > Rs. 5000)\n";
        cout << "-----\n";
        cout << "Total Payable:       Rs. " << totalPayable << "\n";
        cout << "\nEnjoy Your Meal :)\n";
        cout << "=====\n";
    }
    void displayMenu() {
        int choice;
        do {
            cout << "\n===== RESTAURANT MAIN MENU =====\n";
            cout << "1. View Food Menu\n";
            cout << "2. Place Order\n";
            cout << "3. Calculate Bill\n";
            cout << "4. View Customer Details\n";
            cout << "5. Exit\n";
            cout << "Enter your choice: ";
            cin >> choice;
            switch (choice) {
                case 1: displayFoodMenu(); break;
                case 2: placeOrder(); break;
                case 3: displayFinalBill(); break;
                case 4: displayCustomerDetails(); break;
                case 5: cout << "Thank you! Visit Again!\n"; break;
                default: cout << "Invalid choice! Try again.\n";
            }
        } while (choice != 5);
    }
}

int main() {
    cout << "=====\n";
    cout << "    RESTAURANT BILLING SYSTEM        \n";
    cout << "=====\n";
    registerCustomer();
    displayMenu();
    return 0;
}

```

Program 3: Grocery Mart Billing System

```

#include <iostream>
#include <string>
#include <iomanip>
using namespace std;
string customerName, customerId, customerType, paymentMethod;
string groceryItems[] = {"Rice 1 KG", "Sugar 1 KG", "Cooking Oil 1 Litre",
                        "Milk Pack", "Tea Pack", "Flour 5 KG", "Eggs Dozen",
                        "Detergent"};
double groceryPrices[] = {350, 180, 580, 220, 450, 950, 320, 600};
int totalItems = 8;
int cartQty[8] = {0};
void registerCustomer() {
    cout << "\n===== CUSTOMER REGISTRATION =====\n";
    cout << "Enter Customer Name: ";
    cin.ignore();
    getline(cin, customerName);
    cout << "Enter Customer ID: ";
    cin >> customerId;
}

```

```

    cout << "Customer Type (1 = Regular, 2 = Member): ";
    int typeChoice;
    cin >> typeChoice;
    customerType = (typeChoice == 1) ? "Regular" : "Member";
    cout << "Payment Method (1 = Cash, 2 = Card): ";
    int payChoice;
    cin >> payChoice;
    paymentMethod = (payChoice == 1) ? "Cash" : "Card";
    cout << "\nRegistration Successful!\n";
}

void displayGroceryList() {
    cout << "\n===== GROCERY ITEMS =====\n";
    cout << left << setw(5) << "No." << setw(25) << "Item" << "Price\n";
    cout << "-----\n";
    for (int i = 0; i < totalItems; i++) {
        cout << left << setw(5) << (i + 1) << setw(25) << groceryItems[i] << "Rs. " <<
groceryPrices[i] << "\n";
    }
    cout << "=====\n";
}

void addItemToCart() {
    displayGroceryList();
    int itemNo, qty;
    char more;
    do {
        cout << "Enter Item Number: ";
        cin >> itemNo;
        if (itemNo < 1 || itemNo > totalItems) {
            cout << "Invalid item number!\n";
        } else {
            cout << "Enter Quantity: ";
            cin >> qty;
            cartQty[itemNo - 1] += qty;
            cout << qty << "x " << groceryItems[itemNo - 1] << " added to cart.\n";
        }
        cout << "Add more items? (y/n): ";
        cin >> more;
    } while (more == 'y' || more == 'Y');
}

double calculateGrossBill() {
    double total = 0;
    for (int i = 0; i < totalItems; i++) {
        total += cartQty[i] * groceryPrices[i];
    }
    return total;
}

double calculateSalesTax() {
    double foodTotal = 0, nonFoodTotal = 0;
    for (int i = 0; i < 7; i++)
        foodTotal += cartQty[i] * groceryPrices[i];
    nonFoodTotal += cartQty[7] * groceryPrices[7];
    return (foodTotal * 0.05) + (nonFoodTotal * 0.10);
}

double calculateMembershipDiscount(double grossBill) {
    if (customerType == "Member")
        return grossBill * 0.07;
    return 0;
}

double calculateBillAmountDiscount(double amount) {
    if (amount >= 5000 && amount <= 10000)
        return amount * 0.05;
    else if (amount > 10000)
        return amount * 0.10;
    return 0;
}

double calculateCardCharges(double amount) {
    if (paymentMethod == "Card")

```

```

        return amount * 0.02;
    return 0;
}
int calculateLoyaltyPoints(double totalBill) {
    return (int)(totalBill / 100);
}
void displayFinalBill() {
    double grossBill      = calculateGrossBill();
    double salesTax       = calculateSalesTax();
    double memberDiscount = calculateMembershipDiscount(grossBill);
    double billDiscount   = calculateBillAmountDiscount(grossBill);
    double subtotal       = grossBill + salesTax - memberDiscount - billDiscount;
    double cardCharges    = calculateCardCharges(subtotal);
    double totalBill      = subtotal + cardCharges;
    int newLoyaltyPoints  = calculateLoyaltyPoints(totalBill);
    cout << fixed << setprecision(2);
    cout << "\n===== GROCERY MART BILL =====\n";
    cout << "Customer Name:   " << customerName << "\n";
    cout << "Customer Type:   " << customerType << "\n";
    cout << "Payment Method:  " << paymentMethod << "\n\n";
    cout << "---- Items Purchased ----\n";
    for (int i = 0; i < totalItems; i++) {
        if (cartQty[i] > 0)
            cout << groceryItems[i] << " x" << cartQty[i] << " = Rs. " << cartQty[i] *
groceryPrices[i] << "\n";
    }
    cout << "\n";
    cout << "Gross Bill:           Rs. " << grossBill << "\n";
    cout << "Sales Tax:           Rs. " << salesTax << "\n";
    cout << "Membership Discount: Rs. " << memberDiscount << "\n";
    cout << "Bill Discount:       Rs. " << billDiscount << "\n";
    cout << "Card Charges (2%):   Rs. " << cardCharges << "\n";
    cout << "\nGenerated Loyalty Points: " << newLoyaltyPoints << "\n";
    int existingPoints;
    cout << "Enter Your Existing Loyalty Points (0 if new user): ";
    cin >> existingPoints;
    int totalLoyaltyPoints = existingPoints + newLoyaltyPoints;
    cout << "Loyalty Points after Purchase: " << totalLoyaltyPoints << "\n";
    cout << "Press 1 to Redeem Loyalty Points, Press 2 to Continue: ";
    int redeemChoice;
    cin >> redeemChoice;
    double loyaltyDiscount = 0;
    if (redeemChoice == 1) {
        loyaltyDiscount = totalLoyaltyPoints;
        totalBill -= loyaltyDiscount;
        totalLoyaltyPoints = 0;
        cout << "Loyalty Points Redeemed! Discount: Rs. " << loyaltyDiscount << "\n";
    }
    cout << "-----\n";
    cout << "Total Payable Amount: Rs. " << totalBill << "\n";
    cout << "\nThank You For Shopping :)\n";
    cout << "=====\n";
}
void displayCustomerDetails() {
    cout << "\n===== CUSTOMER DETAILS =====\n";
    cout << "Name:           " << customerName << "\n";
    cout << "ID:            " << customerId << "\n";
    cout << "Type:          " << customerType << "\n";
    cout << "Payment Method: " << paymentMethod << "\n";
    cout << "=====\n";
}
void displayMenu() {
    int choice;
    do {
        cout << "\n===== GROCERY MART MAIN MENU =====\n";
        cout << "1. View Grocery Items\n";
        cout << "2. Add Items to Cart\n";
    }
}

```



```

        cout << "3. Calculate Bill\n";
        cout << "4. View Customer Details\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;
        switch (choice) {
            case 1: displayGroceryList(); break;
            case 2: addItemToCart(); break;
            case 3: displayFinalBill(); break;
            case 4: displayCustomerDetails(); break;
            case 5: cout << "Goodbye!\n"; break;
            default: cout << "Invalid choice! Try again.\n";
        }
    } while (choice != 5);
}

int main() {
    cout << "=====\n";
    cout << "      GROCERY MART BILLING SYSTEM      \n";
    cout << "=====\n";
    registerCustomer();
    displayMenu();
    return 0;
}

```

Program 4: Online Shopping System

```

#include <iostream>
#include <string>
#include <iomanip>
using namespace std;
string userName, userEmail, userCity, customerType, paymentMethod;
string products[] = {"T-Shirt", "Jeans", "Shoes", "Watch", "Handbag", "Headphones",
"Mobile Cover", "Perfume"};
double productPrices[] = {1200, 3500, 5000, 2500, 4200, 3000, 700, 2800};
int totalProducts = 8;
int cartQty[8] = {0};
void registerUser() {
    cout << "\n==== USER REGISTRATION =====\n";
    cout << "Enter User Name: ";
    cin.ignore();
    getline(cin, userName);
    cout << "Enter Email: ";
    cin >> userEmail;
    cout << "Enter City: ";
    cin.ignore();
    getline(cin, userCity);
    cout << "Customer Type (1 = New Customer, 2 = Returning Customer): ";
    int typeChoice;
    cin >> typeChoice;
    customerType = (typeChoice == 1) ? "New Customer" : "Returning Customer";
    cout << "\nRegistration Successful!\n";
}

void displayProducts() {
    cout << "\n===== PRODUCT LIST =====\n";
    cout << left << setw(5) << "No." << setw(20) << "Product" << "Price\n";
    cout << "-----\n";
    for (int i = 0; i < totalProducts; i++) {
        cout << left << setw(5) << (i + 1) << setw(20) << products[i] << "Rs. " <<
productPrices[i] << "\n";
    }
    cout << "=====\n";
}

```

```

}
void addToCart() {
    displayProducts();
    int itemNo, qty;
    char more;
    do {
        cout << "Enter Product Number: ";
        cin >> itemNo;
        if (itemNo < 1 || itemNo > totalProducts) {
            cout << "Invalid product number!\n";
        } else {
            cout << "Enter Quantity: ";
            cin >> qty;
            cartQty[itemNo - 1] += qty;
            cout << qty << "x " << products[itemNo - 1] << " added to cart.\n";
        }
        cout << "Add more items? (y/n): ";
        cin >> more;
    } while (more == 'y' || more == 'Y');
}
double calculateProductTotal() {
    double total = 0;
    for (int i = 0; i < totalProducts; i++) {
        total += cartQty[i] * productPrices[i];
    }
    return total;
}
double calculateGST(double amount) {
    return amount * 0.17;
}
double calculateDeliveryCharges() {
    if (userCity == "Lahore" || userCity == "Karachi" || userCity == "Islamabad" ||
        userCity == "lahore" || userCity == "karachi" || userCity == "islamabad")
        return 250;
    else
        return 500;
}
double calculateCustomerDiscount(double productTotal) {
    if (customerType == "New Customer")
        return productTotal * 0.05;
    else
        return productTotal * 0.10;
}
double calculateOrderValueDiscount(double amount) {
    if (amount >= 5000 && amount <= 10000)
        return amount * 0.05;
    else if (amount > 10000)
        return amount * 0.12;
    return 0;
}
double calculatePaymentCharges(double amount) {
    if (paymentMethod == "Card")
        return amount * 0.025;
    return 0;
}
void displayCheckoutBill() {
    cout << "Payment Method (1 = Cash on Delivery, 2 = Debit/Credit Card): ";
    int payChoice;
    cin >> payChoice;
    paymentMethod = (payChoice == 1) ? "Cash on Delivery" : "Card";
    double productTotal = calculateProductTotal();
    double gst = calculateGST(productTotal);
    double deliveryCharges = calculateDeliveryCharges();
    double customerDiscount = calculateCustomerDiscount(productTotal);
    double orderDiscount = calculateOrderValueDiscount(productTotal);
    double subtotal = productTotal + gst + deliveryCharges - customerDiscount -
orderDiscount;
}

```

```

double paymentCharges = calculatePaymentCharges(subtotal);
double finalAmount = subtotal + paymentCharges;
cout << fixed << setprecision(2);
cout << "\n===== ONLINE SHOPPING BILL =====\n";
cout << "User Name: " << userName << "\n";
cout << "City: " << userCity << "\n";
cout << "Customer Type: " << customerType << "\n\n";
cout << "--- Items Ordered ---\n";
for (int i = 0; i < totalProducts; i++) {
    if (cartQty[i] > 0)
        cout << products[i] << " x" << cartQty[i] << " = Rs. " << cartQty[i] *
productPrices[i] << "\n";
}
cout << "\n";
cout << "Product Total: Rs. " << productTotal << "\n";
cout << "GST (17%): Rs. " << gst << "\n";
cout << "Delivery Charges: Rs. " << deliveryCharges << "\n";
cout << "Customer Discount: Rs. " << customerDiscount << "\n";
cout << "Order Discount: Rs. " << orderDiscount << "\n";
cout << "Payment Charges: Rs. " << paymentCharges << "\n";
cout << "-----\n";
cout << "Final Payable: Rs. " << finalAmount << "\n";
cout << "\nThank You for Shopping :)\n";
cout << "=====\n";
}

void displayUserDetails() {
    cout << "\n===== USER DETAILS =====\n";
    cout << "Name: " << userName << "\n";
    cout << "Email: " << userEmail << "\n";
    cout << "City: " << userCity << "\n";
    cout << "Customer Type: " << customerType << "\n";
    cout << "=====\n";
}

void displayMenu() {
    int choice;
    do {
        cout << "\n===== ONLINE SHOPPING MAIN MENU =====\n";
        cout << "1. View Products\n";
        cout << "2. Add Product to Cart\n";
        cout << "3. Calculate Checkout Bill\n";
        cout << "4. View User Details\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;
        switch (choice) {
            case 1: displayProducts(); break;
            case 2: addToCart(); break;
            case 3: displayCheckoutBill(); break;
            case 4: displayUserDetails(); break;
            case 5: cout << "Thank you! Shop Again!\n"; break;
            default: cout << "Invalid choice! Try again.\n";
        }
    } while (choice != 5);
}

int main() {
    cout << "=====\n";
    cout << " ONLINE SHOPPING SYSTEM \n";
    cout << "=====\n";
    registerUser();
    displayMenu();
    return 0;
}

```

Program 5: Social Media Campaign Management System

```
#include <iostream>
#include <string>
#include <iomanip>
using namespace std;
string clientName, businessName, businessType;
int campaignDuration;
string platforms[] = {"Instagram", "Facebook", "LinkedIn"};
double baseCharges[] = {15000, 12000, 20000};
int totalPlatforms = 3;
int selectedPlatform = -1;
void registerClient() {
    cout << "\n==== CLIENT REGISTRATION =====\n";
    cout << "Enter Client Name: ";
    cin.ignore();
    getline(cin, clientName);
    cout << "Enter Business Name: ";
    getline(cin, businessName);
    cout << "Business Type (1 = Small, 2 = Medium, 3 = Corporate): ";
    int typeChoice;
    cin >> typeChoice;
    if (typeChoice == 1) businessType = "Small Business";
    else if (typeChoice == 2) businessType = "Medium Business";
    else businessType = "Corporate Business";
    cout << "Campaign Duration (in days): ";
    cin >> campaignDuration;
    cout << "\nRegistration Successful!\n";
}
void displayPlatforms() {
    cout << "\n==== SOCIAL MEDIA PLATFORMS =====\n";
    cout << left << setw(5) << "No." << setw(15) << "Platform" << "Base Management\n";
    cout << "-----\n";
    for (int i = 0; i < totalPlatforms; i++) {
        cout << left << setw(5) << (i + 1) << setw(15) << platforms[i] << "Rs. " <<
baseCharges[i] << "\n";
    }
    cout << "=====\n";
}
void selectPlatform() {
    displayPlatforms();
    cout << "Select Platform Number: ";
    cin >> selectedPlatform;
    if (selectedPlatform < 1 || selectedPlatform > totalPlatforms) {
        cout << "Invalid selection!\n";
        selectedPlatform = -1;
    } else {
        cout << platforms[selectedPlatform - 1] << " selected.\n";
    }
}
double calculatePostDesignCost(int &staticPosts, int &reelPosts, int &carouselPosts) {
    cout << "\nEnter Number of Static Posts: ";
    cin >> staticPosts;
    cout << "Enter Number of Reel/Video Posts: ";
    cin >> reelPosts;
    cout << "Enter Number of Carousel Posts: ";
    cin >> carouselPosts;
    return (staticPosts * 1000) + (reelPosts * 2500) + (carouselPosts * 1800);
}
double calculateAdHandlingFee(double adBudget) {
    if (adBudget < 50000)
        return adBudget * 0.05;
    else if (adBudget <= 100000)
        return adBudget * 0.08;
```

```

        else
            return adBudget * 0.10;
    }
double calculateExtraDurationCharges() {
    if (campaignDuration > 30)
        return (campaignDuration - 30) * 500;
    return 0;
}
double calculateGST(double amount) {
    return amount * 0.16;
}
double calculateDiscount(double amount) {
    if (businessType == "Small Business")
        return amount * 0.05;
    else if (businessType == "Medium Business")
        return amount * 0.08;
    else
        return amount * 0.10;
}
void displayFinalCampaignBill() {
    if (selectedPlatform == -1) {
        cout << "Please select a platform first!\n";
        return;
    }
    int staticPosts = 0, reelPosts = 0, carouselPosts = 0;
    double postDesignCost = calculatePostDesignCost(staticPosts, reelPosts,
carouselPosts);
    double adBudget = 0;
    cout << "Enter Ad Budget (Rs.): ";
    cin >> adBudget;
    double platformCharges = baseCharges[selectedPlatform - 1];
    double adHandlingFee = calculateAdHandlingFee(adBudget);
    double extraDuration = calculateExtraDurationCharges();
    double subtotal = platformCharges + postDesignCost + adBudget + adHandlingFee
+ extraDuration;
    double gst = calculateGST(subtotal);
    double discount = calculateDiscount(subtotal + gst);
    double finalCost = subtotal + gst - discount;
    cout << fixed << setprecision(2);
    cout << "\n===== SOCIAL MEDIA CAMPAIGN BILL =====\n";
    cout << "Client Name: " << clientName << "\n";
    cout << "Business Name: " << businessName << "\n";
    cout << "Business Type: " << businessType << "\n";
    cout << "Platform: " << platforms[selectedPlatform - 1] << "\n";
    cout << "Campaign Duration: " << campaignDuration << " Days\n\n";
    cout << "Platform Management Charges: Rs. " << platformCharges << "\n";
    cout << "Post Design Cost: Rs. " << postDesignCost << "\n";
    cout << "Ad Budget: Rs. " << adBudget << "\n";
    cout << "Ad Handling Fee: Rs. " << adHandlingFee << "\n";
    cout << "Extra Duration Charges: Rs. " << extraDuration << "\n";
    cout << "GST (16%): Rs. " << gst << "\n";
    cout << "Discount: Rs. " << discount << "\n";
    cout << "-----\n";
    cout << "Final Campaign Cost: Rs. " << finalCost << "\n";
    cout << "=====\n";
}
void displayClientDetails() {
    cout << "\n===== CLIENT DETAILS =====\n";
    cout << "Client Name: " << clientName << "\n";
    cout << "Business Name: " << businessName << "\n";
    cout << "Business Type: " << businessType << "\n";
    cout << "Duration: " << campaignDuration << " days\n";
    cout << "=====\n";
}
void displayMenu() {
    int choice;
    do {

```

```

        cout << "\n===== SOCIAL MEDIA CAMPAIGN MENU =====\n";
        cout << "1. View Platforms\n";
        cout << "2. Select Campaign Platform\n";
        cout << "3. Calculate Campaign Cost\n";
        cout << "4. View Client Details\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;
        switch (choice) {
            case 1: displayPlatforms(); break;
            case 2: selectPlatform(); break;
            case 3: displayFinalCampaignBill(); break;
            case 4: displayClientDetails(); break;
            case 5: cout << "Goodbye!\n"; break;
            default: cout << "Invalid choice! Try again.\n";
        }
    } while (choice != 5);
}

int main() {
    cout << "=====\n";
    cout << "  SOCIAL MEDIA CAMPAIGN MANAGEMENT      \n";
    cout << "===== \n";
    registerClient();
    displayMenu();
    return 0;
}

```

Program 6: Personal Student Diary

```

#include <iostream>
#include <string>
#include <ctime>
#include <iomanip>
using namespace std;
const int MAX_ENTRIES = 20;
struct DiaryEntry {
    string title;
    string content;
    string createdAt;
    string lastModified;
};
DiaryEntry diary[MAX_ENTRIES];
int entryCount = 0;
string loggedInUser = "";
string storedUsername = "";
string storedPassword = "";
bool isRegistered = false;
string getCurrentDateTime() {
    time_t now = time(0);
    char buf[25];
    strftime(buf, sizeof(buf), "%Y-%m-%d %H:%M:%S", localtime(&now));
    return string(buf);
}
void registerUser() {
    if (isRegistered) {
        cout << "An account already exists. Please login.\n";
        return;
    }
    cout << "\n===== REGISTER =====\n";
    cout << "Enter Username: ";
    cin >> storedUsername;
}

```

```

        cout << "Enter Password: ";
        cin >> storedPassword;
        isRegistered = true;
        cout << "Registration successful! You can now login.\n";
    }
    bool loginUser() {
        if (!isRegistered) {
            cout << "No account found. Please register first.\n";
            return false;
        }
        cout << "\n===== LOGIN =====\n";
        string username, password;
        cout << "Enter Username: ";
        cin >> username;
        cout << "Enter Password: ";
        cin >> password;
        if (username == storedUsername && password == storedPassword) {
            loggedInUser = username;
            cout << "Login successful! Welcome, " << username << "!\n";
            return true;
        } else {
            cout << "Invalid username or password!\n";
            return false;
        }
    }
    void createEntry() {
        if (entryCount >= MAX_ENTRIES) {
            cout << "Diary is full! Maximum 20 entries allowed.\n";
            return;
        }
        cout << "\n===== CREATE NEW ENTRY =====\n";
        string title, content;
        cin.ignore();
        cout << "Enter Title: ";
        getline(cin, title);
        if (title.empty()) {
            cout << "Title cannot be empty!\n";
            return;
        }
        cout << "Enter Content: ";
        getline(cin, content);
        diary[entryCount].title = title;
        diary[entryCount].content = content;
        diary[entryCount].createdAt = getCurrentDateTime();
        diary[entryCount].lastModified = diary[entryCount].createdAt;
        entryCount++;
        cout << "Entry created successfully!\n";
    }
    void viewAllEntries() {
        if (entryCount == 0) {
            cout << "No diary entries found.\n";
            return;
        }
        cout << "\n===== ALL DIARY ENTRIES (" << entryCount << " total) =====\n";
        cout << left << setw(5) << "No." << setw(30) << "Title" << "Created At\n";
        cout << "-----\n";
        for (int i = 0; i < entryCount; i++) {
            cout << left << setw(5) << (i + 1)
                << setw(30) << diary[i].title
                << diary[i].createdAt << "\n";
        }
        cout << "=====\n";
    }
    void readFullEntry() {
        viewAllEntries();
        if (entryCount == 0) return;
        int entryNo;

```

```

    cout << "Enter Entry Number to Read: ";
    cin >> entryNo;
    if (entryNo < 1 || entryNo > entryCount) {
        cout << "Invalid entry number!\n";
        return;
    }
    int idx = entryNo - 1;
    cout << "\n===== DIARY ENTRY #" << entryNo << " =====\n";
    cout << "Title:          " << diary[idx].title << "\n";
    cout << "Created:           " << diary[idx].createdAt << "\n";
    cout << "Last Modified:    " << diary[idx].lastModified << "\n";
    cout << "-----\n";
    cout << diary[idx].content << "\n";
    cout << "=====\\n";
}

void editEntry() {
    viewAllEntries();
    if (entryCount == 0) return;
    int entryNo;
    cout << "Enter Entry Number to Edit: ";
    cin >> entryNo;
    if (entryNo < 1 || entryNo > entryCount) {
        cout << "Invalid entry number!\n";
        return;
    }
    int idx = entryNo - 1;
    cin.ignore();
    cout << "Current Title: " << diary[idx].title << "\n";
    cout << "Enter New Title (press Enter to keep current): ";
    string newTitle;
    getline(cin, newTitle);
    if (!newTitle.empty())
        diary[idx].title = newTitle;
    cout << "Current Content:\n" << diary[idx].content << "\n";
    cout << "Enter New Content (press Enter to keep current): ";
    string newContent;
    getline(cin, newContent);
    if (!newContent.empty())
        diary[idx].content = newContent;
    diary[idx].lastModified = getCurrentDateTime();
    cout << "Entry updated successfully!\n";
}

void deleteOneEntry() {
    viewAllEntries();
    if (entryCount == 0) return;
    int entryNo;
    cout << "Enter Entry Number to Delete: ";
    cin >> entryNo;
    if (entryNo < 1 || entryNo > entryCount) {
        cout << "Invalid entry number!\n";
        return;
    }
    for (int i = entryNo - 1; i < entryCount - 1; i++) {
        diary[i] = diary[i + 1];
    }
    entryCount--;
    cout << "Entry deleted successfully!\n";
}

void deleteAllEntries() {
    if (entryCount == 0) {
        cout << "No entries to delete.\n";
        return;
    }
    cout << "Are you sure you want to delete ALL entries? (y/n): ";
    char confirm;
    cin >> confirm;
    if (confirm == 'y' || confirm == 'Y') {

```



```

        entryCount = 0;
        cout << "All entries deleted.\n";
    } else {
        cout << "Operation cancelled.\n";
    }
}

void searchByKeyword() {
    if (entryCount == 0) {
        cout << "No entries to search.\n";
        return;
    }
    cin.ignore();
    string keyword;
    cout << "Enter keyword to search in titles: ";
    getline(cin, keyword);
    bool found = false;
    cout << "\n===== SEARCH RESULTS =====\n";
    for (int i = 0; i < entryCount; i++) {
        string titleLower = diary[i].title;
        string keyLower = keyword;
        for (char &c : titleLower) c = tolower(c);
        for (char &c : keyLower) c = tolower(c);
        if (titleLower.find(keyLower) != string::npos) {
            cout << (i + 1) << ". " << diary[i].title << " [" << diary[i].createdAt <<
"]\n";
            found = true;
        }
    }
    if (!found)
        cout << "No entries found with keyword: " << keyword << "\n";
    cout << "===== \n";
}

void diaryMenu() {
    int choice;
    do {
        cout << "\n===== PERSONAL DIARY MENU =====\n";
        cout << "Total Entries: " << entryCount << " / " << MAX_ENTRIES << "\n";
        cout << "1. Create New Diary Entry\n";
        cout << "2. View All Entries\n";
        cout << "3. Read Full Entry\n";
        cout << "4. Edit an Entry\n";
        cout << "5. Delete One Entry\n";
        cout << "6. Delete All Entries\n";
        cout << "7. Search Entry by Keyword (Bonus)\n";
        cout << "8. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;
        switch (choice) {
            case 1: createEntry(); break;
            case 2: viewAllEntries(); break;
            case 3: readFullEntry(); break;
            case 4: editEntry(); break;
            case 5: deleteOneEntry(); break;
            case 6: deleteAllEntries(); break;
            case 7: searchByKeyword(); break;
            case 8: cout << "Goodbye, " << loggedInUser << "!\n"; break;
            default: cout << "Invalid choice! Try again.\n";
        }
    } while (choice != 8);
}

void authMenu() {
    int choice;
    bool loggedIn = false;
    do {
        cout << "\n===== STUDENT DIARY - AUTH =====\n";
        cout << "1. Register\n";
        cout << "2. Login\n";
    }
}

```

```

        cout << "3. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;
        switch (choice) {
            case 1: registerUser(); break;
            case 2:
                loggedIn = loginUser();
                if (loggedIn) diaryMenu();
                break;
            case 3: cout << "Goodbye!\n"; return;
            default: cout << "Invalid choice!\n";
        }
    } while (true);
}

int main() {
    cout << "=====\n";
    cout << "        PERSONAL STUDENT DIARY        \n";
    cout << "=====\n";
    authMenu();
    return 0;
}

```